Containers for Dummies

Or: Strictly Positive Data Types à la Carte

Jacob Trevor[1]

Today

[1]School of Computing Science, University of Glasgow

The menu:

- Containers are *very useful*, particularly if you are working in a mechanised setting.

The menu:

- Containers are *very useful*, particularly if you are working in a mechanised setting.
- All the literature is category flavoured,

The menu:

- Containers are *very useful*, particularly if you are working in a mechanised setting.
- All the literature is category flavoured,
  - & consequently a touch confusing for some

The menu:

- Containers are *very useful*, particularly if you are working in a mechanised setting.
- All the literature is category flavoured,
  - & consequently a touch confusing for some
- I will explain what containers are, by looking at an application to a particular example
- Hopefully without any reference to the categorical underpinnings
- which is just a well because

The menu:

- Containers are *very useful*, particularly if you are working in a mechanised setting.
- All the literature is category flavoured,
  - & consequently a touch confusing for some
- I will explain what containers are, by looking at an application to a particular example
- Hopefully without any reference to the categorical underpinnings
- which is just a well because

Disclaimer: I am not a category theorist

Quick preliminary: Dependent Type theory

```
def Fin n := { x : Nat // x < n }
-- "all natural numbers less than n"

example : (Σ x : Nat, Fin x) := ⟨1, 0⟩
--          ^ sigma type           ^  ^ Fin x ( = fin 1)
--            (dependent pair)     | x : Nat
-- pair of numbers; 2nd < 1st
```

A motivating example: Expressions

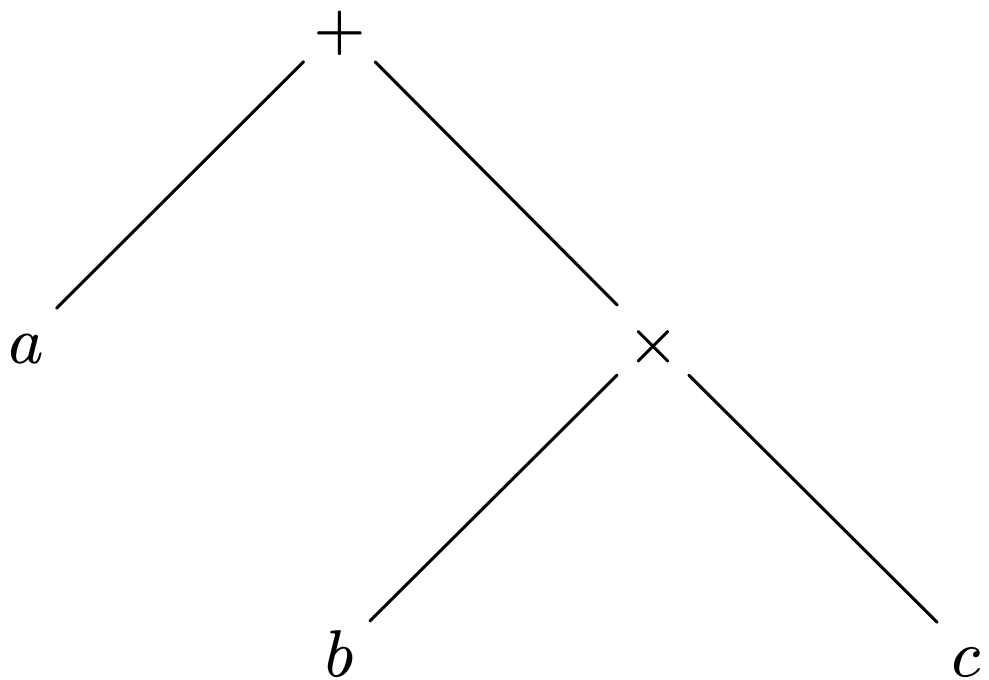
Syntax trees

PL people formalise *languages* as syntax trees

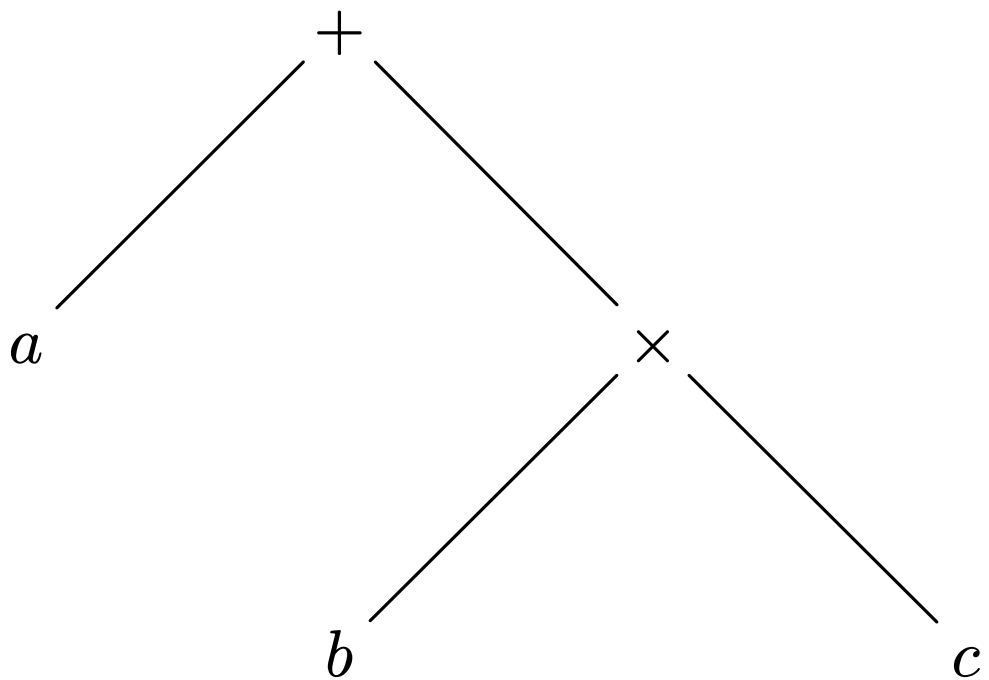
When we write something like

$$a + b \times c$$

We represent it with a tree like this:



We represent it with a tree like this:



We call this an *abstract syntax tree (AST)*

Grammar

We formally describe how to build such trees with a *grammar*

Grammar

We formally describe how to build such trees with a *grammar*

Here's a simple grammar for arithmetic expressions, such as the one we just saw:

$$\begin{aligned} \text{Expr } E = \mathbb{N} \\ \mid E + E \\ \mid E \times E \end{aligned}$$

What if I want to add a new binary operation?

Simple Approach: add more constructors

$$\begin{aligned} \text{Expr } E = \mathbb{N} \\ \mid E + E \\ \mid E \times E \\ \mid E ** E \end{aligned}$$

Simple Approach: add more constructors

$$\begin{aligned} \text{Expr } E = \mathbb{N} \\ \mid E + E \\ \mid E \times E \\ \mid E ** E \end{aligned}$$

- But now, all our old use sites will break
- Need to rewrite everything!
- with this approach, it's difficult to *scope changes*.
- **Can we do better?**

The Expression Problem

“The goal is to define a datatype by cases, where one can add new cases to the datatype and new functions over the datatype, without recompiling existing code, and while retaining static type safety (e.g., no casts).”

> Phil Wadler

Enter Data Types a la Carte

- Fantastic Functional Perl by Wouter Swierstra[1]
  - It's friendly, entertaining and approachable (go read it!)

Basic idea:

1. define *pieces* or *fragments* of the language
2. define a way to glue them together

Step 1: Fragments

$$\begin{aligned}
\text{LitExpr} &= \mathbb{N} \\
\text{AddExpr} &= E + E \\
\text{MulExpr} &= E \times E \\
\text{PowExpr} &= E ** E
\end{aligned}$$

Or, in lean:

```
inductive LitExpr where
  | val : Nat → LitExpr

inductive AddExpr where
  | plus : E → E → AddExpr

inductive MulExpr where
  | mul : E → E → MulExpr

inductive PowExpr where
  | pow : E → E → PowExpr
```

What's `E`?

```
inductive AddExpr where
  | plus : E → E → AddExpr
```

- Here, `E` is *whatever goes in the holes* of (+)
- So, `E` is an expression.

What's `E`?

```
inductive AddExpr where
  | plus : E → E → AddExpr
```

- Here, `E` is *whatever goes in the holes* of (+)
- So, `E` is an expression.
- A simple idea:

```
inductive AddExpr where
  | plus : AddExpr → AddExpr → AddExpr
```

But now, I can only add other add expressions…

What's `E`?

The real fix is to make `E` an argument of the inductive type:

```
inductive AddExpr (E : Type) where
  | plus : E → E → AddExpr E
```

- So an `(AddExpr Nat)` is an add expression where the holes are filled with `Nats`
  - eg. 1 + 2

Important difference

- Before, `Expr` was a type
- but `AddExpr` &c are now *type formers*
  - they need an argument
- *E* is the *type of subterms*
  - this is the expression AS*Tree*
  - E is the type of *child nodes*

Step 2: Glue

- We want “Either Add or Mul or Pow”

Step 2: Glue

- We want “Either Add or Mul or Pow”
  - Sounds like a coproduct!
  - Needs to be a bit special, to pass through `E`

Step 2: Glue

- We want "Either Add or Mul or Pow"
  - Sounds like a coproduct!
  - Needs to be a bit special, to pass through `E`

```
inductive coprod (A B : Type → Type) (E : Type) : Type  where
  | inl : A E
  | inr : B E

infixr:50 " :+: " ⇒ coprod
```

N.B. as a result, `LitExpr` also needs to take in an `E`

What should `E` be?

We can now write:

```
def Expr := LitExpr :+: AddExpr :+: MulExpr :+: PowExpr
```

What should `E` be?

We can now write:

```
def Expr := LitExpr :+: AddExpr :+: MulExpr :+: PowExpr
```

- But this isn't quite right - `Expr` here has type `Type → Type`

What should `E` be?

We can now write:

```
def Expr := LitExpr :+: AddExpr :+: MulExpr :+: PowExpr
```

- But this isn't quite right - `Expr` here has type `Type → Type`
- Recall: `E` is the *type of subterms*

What should `E` be?

We can now write:

```
def Expr := LitExpr :+: AddExpr :+: MulExpr :+: PowExpr
```

- But this isn't quite right - `Expr` here has type `Type → Type`
- Recall: `E` is the *type of subterms*
  - The children of an expression are expressions
- So we want something like:

```
def Expr :=
  (LitExpr :+: AddExpr :+: MulExpr :+: PowExpr) Expr
```

Taking the Fixpoint

Swierstra sets this up with a special function:

```
def fix f := f (fix f)
```

And then you can simply do:

```
def Expr :=  fix (AddExpr :+: MulExpr :+: PowExpr)
```

And just like that, you have defined a totally modular AST for your expressions!

Isn't this attractive?

- Define language in fragments
- Glue them together as you like
- There's also discussion in the paper about how to modularise things like evaluation and type checking
  - in short, use typeclasses

I certainly thought so

Only one problem…

This presentation is badly behaved

```
def fix f := f (fix f)
```

- `fix` doesn't terminate

This presentation is badly behaved

```
def fix f := f (fix f)
```

- `fix` doesn't terminate
  - think about unfolding this definition:

```
def fix f := f (f (f (f ...)))
```

This presentation is badly behaved

```
def fix f := f (fix f)
```

- `fix` doesn't terminate
  - think about unfolding this definition:

```
def fix f := f (f (f (f ...)))
```

- There's no base case for the recursion
- The same thing happens if we use the raw definition for `Expr`

This presentation is badly behaved

```
def fix f := f (fix f)
```

- `fix` doesn't terminate
  - think about unfolding this definition:

```
def fix f := f (f (f (f ...)))
```

- There's no base case for the recursion
- The same thing happens if we use the raw definition for `Expr`
- **It's not allowed in lean**

So, what do we do about it?

Our hero: Containers

Templates ≠ Containers

- Jumping straight in to a full description of containers is probably a bit much
- So I will introduce a simplified version first
- We will slowly hack away at it until we get to the real thing

Let's look at our modular expressions again:

```
inductive LitExpr (E : Type) where
  | val : Nat → LitExpr

inductive AddExpr (E : Type) where
  | plus : E → E → AddExpr

inductive MulExpr (E : Type) where
  | mul : E → E → MulExpr

inductive PowExpr (E : Type) where
  | pow : E → E → PowExpr
```

Expr = data + arity

We can think of each expression as having a *constant part* and an *arity*

Expr = data + arity

We can think of each expression as having a *constant part* and an *arity*

```
inductive AddExpr (E : Type) where
  | plus : E → E → AddExpr
--  ^      ^---^ 2 subterms (Es) = arity 2
--  | constant part = "it's a (+) expr"
```

Expr = data + arity

We can think of each expression as having a *constant part* and an *arity*

```
inductive AddExpr (E : Type) where
  | plus : E → E → AddExpr
--  ^      ^---^ 2 subterms (Es) = arity 2
--  | constant part = "it's a (+) expr"

inductive LitExpr (E : Type) where
  | val : Nat → LitExpr -- 0 subterms (Es) = arity 0
--  ^     ^  constant part = 'its a literal' + value
```

Expr = data + arity

We can think of each expression as having a *constant part* and an *arity*

```
inductive AddExpr (E : Type) where
  | plus : E → E → AddExpr
--  ^      ^---^ 2 subterms (Es) = arity 2
--  | constant part = "it's a (+) expr"

inductive LitExpr (E : Type) where
  | val : Nat → LitExpr -- 0 subterms (Es) = arity 0
--  ^     ^  constant part = 'its a literal' + value
```

The same goes for both of our other expression constructors

Expression Templates

We can define a structure which represents a template for an expression:

```
structure Template where
  data : Type -- some constant part
  arity : Nat -- the arity i.e. #subterms
```

Example:

```
-- the constant data:
inductive LitD where | val   : Nat → LitD
inductive AddD where | plus  : AddD
inductive MulD where | times : MulD
inductive PowD where | pow   : PowD

-- the templates:
def LitExpr : Template := {data := LitD, arity := 0}
def AddExpr : Template := {data := AddD, arity := 2}
def MulExpr : Template := {data := MulD, arity := 2}
def PowExpr : Template := {data := PowD, arity := 2}
```

Expression Instances

We can create an *instance* of a template:

```
structure Template.instance (T : Template) where
  -- What should go here?
```

Expression Instances

We can create an *instance* of a template:

```
structure Template.instance (T : Template) where
  out : T.data × _
--- obviously, we need the "constant part" of the expression
```

Expression Instances

We can create an *instance* of a template:

```
structure Template.instance (T : Template) where
  out : T.data × (Fin T.arity → ?)
-- And something to put in each hole
```

Expression Instances

Given a template, we can create a real instance of that expression:

```
structure Template.instance (T : Template) (E : Type) where
  out : T.data × (Fin T.arity → E)
-- we reuse the "E" trick for the child type
```

Think “A `T`-template with the holes filled with `E`s”

example:

```
inductive LitD where | val : Nat → LitD

def LitExpr : Template := {data := LitD, arity := 0}

def lit1 : LitExpr.instance () -- literal with value 1
  := ⟨.val 1 , fun x ⇒ nomatch x⟩
--                    ^ Fin 0 = an empty type
--                      no nat < 0
```

So to reiterate:

- A template specifies a type of expression via:
  - The type of the *constant part* (data)
  - Some *number of holes* (arity)
- An *instance* of a template is a pair:
  - The actual constant part
  - A way to fill in the holes (in the form of a function)

Fixedpoints

- Now, we can also consider “What if the holes were filled with more expressions?”
- And we get the following construction:

```
structure Template.fix (T : Template) where
  out : T.data × (Fin T.arity → (T.fix))
```

Fixedpoints

- Now, we can also consider “What if the holes were filled with more expressions?”
- And we get the following construction:

```
structure Template.fix (T : Template) where
  out : T.data × (Fin T.arity → (T.fix))
```

Intuivitely: “A T-template with the holes filled with more T-templates”

Fixedpoints

- Now, we can also consider “What if the holes were filled with more expressions?”
- And we get the following construction:

```
structure Template.fix (T : Template) where
  out : T.data × (Fin T.arity → (T.fix))
```

This definition is strictly positive!

→ This basic trick is what makes containers ‘magic’

What about glue?

We want to be able to define a sensible notion of *coproduct* on templates

- Question for the audience - why on the template, not on the instance?

What about glue?

We want to be able to define a sensible notion of *coproduct* on templates

- Question for the audience - why on the template, not on the instance?
- Ans: To get our AST, we define a *template* for an expression and then take a fixedpoint

How do we glue templates together?

Template Coproduct

```
structure Template where
  data : Type
  arity : Nat

def Template.coprod (A B : Template) : Template := _
-- function : Template → Template → Template
```

Template Coproduct

```
structure Template where
  data : Type
  arity : Nat

def Template.coprod (A B : Template) : Template := {
    data := _,
    arity := _,
  }
```

Template Coproduct

```
structure Template where
  data : Type
  arity : Nat

def Template.coprod (A B : Template) : Template := {
    data := A.data ⊕ B.data, -- just use the normal coprod
    arity := _,
  }
```

Template Coproduct

```
structure Template where
  data : Type
  arity : Nat

def Template.coprod (A B : Template) : Template := {
    data := A.data ⊕ B.data, -- just use the normal coprod
    arity := _,
  }
```

- But the arity should depend on *which* kind of expression it is
- A little tweak required

Template Coproduct

```
structure Template where
  data : Type
  arity : data → Nat -- arity now depends on the const data

def Template.coprod (A B : Template) : Template := {
    data := A.data ⊕ B.data,
    arity d := match d with -- just use the matching arity
      | .inl x ⇒ A.arity x
      | .inr x ⇒ B.arity x
  }
```

This requires a litle tweak to the expressions:

```
def LitExpr : Template := {data := LitD, arity := fun _ ⇒ 0}
def AddExpr : Template := {data := AddD, arity := fun _ ⇒ 2}
def MulExpr : Template := {data := MulD, arity := fun _ ⇒ 2}
def PowExpr : Template := {data := PowD, arity := fun _ ⇒ 2}
```

- Note the function for arity
  - we just throw away the arg, since the arity of all these is constant

And to instance and fix:

```
structure Template where
  data : Type
  arity : data → Nat

structure Template.instance (T : Template) (E : Type) where
  out : Σ (x : T.data), (Fin (T.arity x) → E)
```

Why not just roll `Fin` into "arity"?

```
structure Template where
  data : Type
  arity : data → Type

def LitExpr : Template
  := {data := LitD, arity := fun _ ⇒ Fin 0}

def Template.instance (T : Template) (E : Type) :=
  ∑ (x : T.data), (T.arity x → E)
```

Congratulations - you just invented containers:

```
structure Container where
  shape : Type
  pos   : shape → Type

def Container.extension (C : Container) (E : Type) :=
  ∑ x : C.shape, (C.pos x → E)

def Container.fix (C : Container) :=
  ∑ x : C.shape, (C.pos x → T.fix)
```

Anatomy of a container

- If you understood up til this point, then you have (in effect) understood containers
- A container is defined by just two things:
  - a type of *shapes* (this is the constant data)
  - a function from shapes into a type, called the position function (this is the arity)

Anatomy of a container

```
structure Container : Type where
  shape : Type
  pos   : shape → Type
```

This is often written

$$S \triangleleft P$$

(Except in the first paper, where the triangle is reversed…)

Putting things inside

The container object itself doesn't actually *contain* anything yet. For that, we need to take the *extension* of the container:

$$[\![S \lhd P]\!] : \mathrm{Type} \to \mathrm{Type}$$

This is just like an *instance* of the *templates* from earlier.

Putting things inside

The container object itself doesn't actually *contain* anything yet. For that, we need to take the *extension* of the container:

$$[\![S \lhd P]\!] : \text{Type} \to \text{Type}$$

- The extension is a function that takes a type to a type.
- The input is the type of *elements* inside the container.
  - It's `E` (the child type) from earlier
  - "What do you want to put inside?"

Putting things inside

It's defined like this:

$$[\![S \lhd P]\!]E := \Sigma\ s : S, (Ps \to E)$$

Putting things inside

It's defined like this:

$$[\![S \lhd P]\!]E := \Sigma\ s : S, (Ps \to E)$$

- The first element of the pair is a *shape*
  - i.e. constant data

Putting things inside

It's defined like this:

$$[\![S \lhd P]\!]E := \Sigma\ s : S, (Ps \to E)$$

- The first element of the pair is a *shape*
  - i.e. constant data
- The second element is a function from the positions of the shape into the values
  - i.e. for each hole, something to put inside

An example: Lists

```
def list : Container := {
    shape := Nat, pos := fun n ⇒ Fin n
}
```

How do we interpret this?

- The *shape* here is the *length* of the list
- the position function takes a natural number to a Type
  - In this case, `Fin n`

List continued

- Let's fill the list with Natural numbers:

```
def listEx : list.extension Nat
  := ⟨cast (by simp) 3, fun x ⇒ match x with
    | 0 ⇒ 1
    | 1 ⇒ 2
    | 2 ⇒ 4
  ⟩
```

This corresponds to the list `[1, 2, 4]`.

Question: What data type corresponds to the fixpoint of `list`?

Squiggly Horrors

Polynomial functors

- Extension $[\![-]\!]$ is a functor
  - e.g. concretely, in the category of types and total functions
  - Extension takes types $(\alpha : \text{Type})$ to types $([\![S \lhd P]\!]\alpha : \text{Type})$
  - and functions $f : \alpha \to \beta$ to functions
    $[\![S \lhd P]\!]f : [\![S \lhd P]\!]\alpha \to [\![S \lhd P]\!]\beta$
  - "Just map over the children"
- These functors are 'nice' (i.e. polynomial)

Polynomial functors?

$$[\![S \lhd P]\!]X := \Sigma\ s : S, (Ps \to X)$$

- $(A \to B)$ is a type
- Sometimes called the "function space" of A to B
- The cardinality of this type is B^A
- So sometimes we write B^A for the function space
- So we rewrite our expressions at the top to...

Polynomial functors?

$$[\![S \lhd P]\!]X := \Sigma\ s : S, X^{Ps}$$

- Sigma types = sum over possible values in a type
- function spaces = generalised exponents

Sound familiar?

$$\mathbb{Z}[x] := \sum_k \alpha_k x^k$$

- All containers are polynomial functors; all polynomial functors are containers.

Initial algebra

- We can always find the initial algebra for a polynomial functor
- This gives us a convenient, rigorous way to handle recursion
  - Containers are great ways of setting up inductive data types
  - Useful backbone for theorem proving
- Come back next week if you want to know what that means!

Algebra

- Containers define a *signature* for a singly-kinded algebra
  - Shapes with no holes are *constants*
  - Shapes with n holes are n-ary operators
- Indexed containers can handle multi-kinded algebras
- Directed containers model coalgebras
- non-free algebras (i.e. algebras with axioms) can be handled by *quotient containers*

$\rightarrow$ highly relevant to my work; come and chat if you want to know more

In conclusion:

- Containers are extremely useful
  - Can model anything where there's a set of questions, and for each, a set of allowable responses
  - If you can phrase your data as a container, it *will* be OK in lean/agda/rocq
- They allow you to constructively model fixed points

Bibliography

[1] W. Swierstra, “Data types à la carte,” *J. Funct. Program.*, vol. 18, no. 4, pp. 423–436, 2008, doi: 10.1017/S0956796808006758.

Thanks for your attention!

Any questions?